Fundamentos de Informática

Matrices: arreglos bidimensionales

La tabla del TP ya es una matriz

- **Fila** = una prueba (ensayo)
- **Columna** = una variable medida: presión, caudal, temperatura, potencia
- Todos los valores son del mismo tipo (números reales) → una matriz de `float`
- En C++, una matriz es un arreglo de arreglos (cada fila es un arreglo)

Presión · Caudal · Temp. · Potencia

	j=0	j=1	j=2	j=3
i=0	2.1	17.0	32	5.3
i=1	2.8	14.1	37	5.8
i=2	2.3	16.2	33	5.4
i=3	3.2	17.7	38	6.7
⋮	⋮	⋮	⋮	⋮
i=19	3.2	16.1	40	6.2

20 × 4

= 80 valores `float` en memoria, uno al lado del otro, fila por fila

Declarar una matriz

```
float datos[20][4];
```

`float`	`datos`	`[20]`	`[4]`
tipo de cada elemento	nombre de la matriz	cantidad de filas	cantidad de columnas

```
const int FILAS = 20;
const int COLUMNAS = 4;

float datos[FILAS][COLUMNAS];
```

- Buena práctica: usar constantes para los tamaños
- Si cambia la cantidad de pruebas, se modifica en un solo lugar
- El tamaño debe conocerse al compilar

Inicializar con los datos del ensayo

```
float datos[FILAS][COLUMNAS] = {
  // Pres. Caudal  Temp.  Pot.
  { 2.1f, 17.0f, 32.0f, 5.3f }, // P1
  { 2.8f, 14.1f, 37.0f, 5.8f }, // P2
  { 2.3f, 16.2f, 33.0f, 5.4f }, // P3
  // ... 16 filas más ...
  { 3.2f, 16.1f, 40.0f, 6.2f }  // P20
};
```

- Cada par de llaves internas **`{ }`** es una fila
- Las filas se separan con comas; la última no lleva
- El sufijo **`f`** indica que el número es **`float`**
- Los valores que falten quedan en 0: **`float m[3][4] = {};`**
- La matriz completa ya está cargada en el archivo **`bombeo_matriz.cpp`**

Acceder a un elemento: `datos[i][j]`

	j=0	j=1	j=2	j=3
i=0	**2.1**	17.0	32	5.3
i=1	2.8	14.1	37	5.8
i=2	2.3	16.2	33	5.4
i=3	3.2	17.7	38	6.7
i=4	2.7	13.8	36	5.5
i=5	1.9	15.8	31	4.8
i=6	3.4	16.1	**42**	6.3
⋮	⋮	⋮	⋮	⋮
i=19	3.2	16.1	40	**6.2**

```
datos[0][0]  // 2.1 presión, prueba 1
datos[6][2]  // 42  temp., prueba 7
datos[19][3] // 6.2 potencia, prueba 20

datos[0][0] = 2.2f;  // modificar
```

Los índices empiezan en 0

Filas válidas: 0 a 19 · columnas válidas: 0 a 3

La prueba N está en la fila N − 1

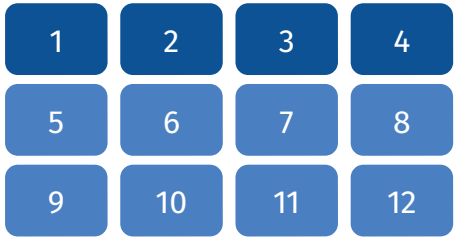
Recorrer toda la matriz: dos `for` anidados

```
for (int i = 0; i < FILAS; i++) {
    for (int j = 0; j < COLUMNAS; j++) {
        cout << datos[i][j] << "\t";
    }
    cout << endl;   // fin de la fila
}
```

- **El `for` de afuera** elige la fila `i`
- **El `for` de adentro** recorre las columnas `j` de esa fila
- La tabulación `"\t"` separa los valores de la fila; `endl` pasa a la fila siguiente

Salida

```
2.1     17      32      5.3
2.8     14.1    37      5.8
...
```



Orden de visita: fila por fila

Recorrer una sola columna o una sola fila

Columna fija (una variable): `j` fijo, `i` varía

```
int j = 1;  // caudal
float maximo = datos[0][j];
for (int i = 1; i < FILAS; i++) {
    if (datos[i][j] > maximo) {
        maximo = datos[i][j];
    }
}
```

Fila fija (una prueba): `i` fijo, `j` varía

```
int i = 6;  // prueba 7
for (int j = 0; j < COLUMNAS; j++) {
    cout << datos[i][j] << " ";
}
```

Patrón acumulador: una variable que arranca con un valor inicial y se actualiza en cada vuelta del **`for`**. Sirve para máximos, mínimos, conteos y sumas. Para analizar una variable del ensayo, el recorrido es por columna.

Guardar resultados en otros arreglos

```
// diferencia de cada valor respecto
// de la prueba 1 (misma forma 20 x 4)
float dif[FILAS][COLUMNAS];
for (int i = 0; i < FILAS; i++) {
    for (int j = 0; j < COLUMNAS; j++) {
        dif[i][j] = datos[i][j] - datos[0][j];
    }
}
```

- No pisar los datos originales: crear un arreglo nuevo para cada resultado
- Un vector (1 dimensión) también es útil: **`float v[4]`** guarda un valor por columna

¿Qué forma tiene el resultado?

Un valor por elemento	`float r[20][4]`
Un valor por variable	`float v[4]`
Un valor por prueba	`float p[20]`
Variable contra variable	`float m[4][4]`

Transponer: las filas pasan a ser columnas

`datos` 20 filas × 4 columnas

	j=0	j=1	j=2	j=3
i=0	2.1	17.0	32	5.3
i=1	2.8	14.1	**37**	5.8
i=2	2.3	16.2	33	5.4
⋮	⋮	⋮	⋮	⋮
i=19	3.2	16.1	40	6.2



transponer

`t` 4 filas × 20 columnas

	i=0	i=1	i=2	…	i=19
j=0 Presión	2.1	2.8	2.3	…	3.2
j=1 Caudal	17.0	14.1	16.2	…	16.1
j=2 Temp.	32	**37**	33	…	40
j=3 Potencia	5.3	5.8	5.4	…	6.2

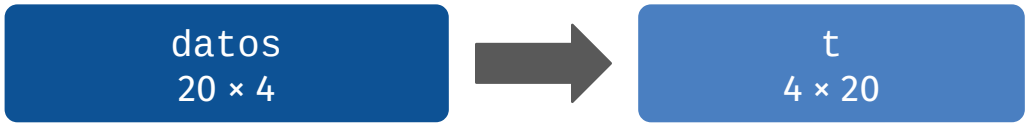
Cada columna de **`datos`** se convierte en una fila de **`t`** (mismo color).

El elemento resaltado: **`datos[1][2] = 37 → t[2][1] = 37`**

Regla: `datos[i][j]` pasa a `t[j][i]` (se intercambian `i` y `j`).

Transponer en código

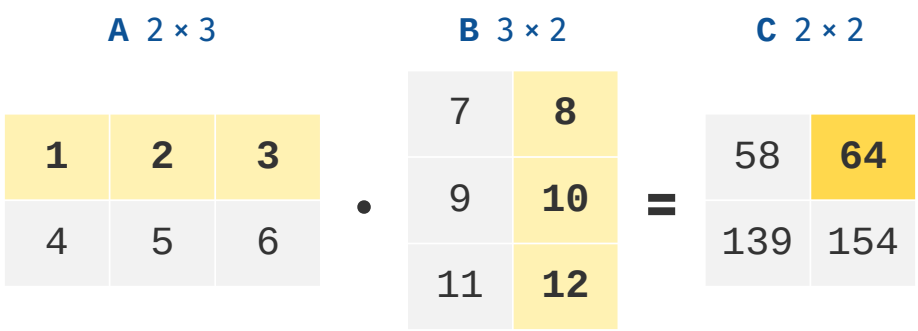
```
float t[COLUMNAS][FILAS];   // 4 x 20
for (int i = 0; i < FILAS; i++) {
    for (int j = 0; j < COLUMNAS; j++) {
        t[j][i] = datos[i][j];
    }
}
```



La matriz destino se declara con las dimensiones invertidas.

- Se recorre la matriz original con el doble **`for`** de siempre
- Lo único nuevo es el orden de los índices: **`t[j][i]`**
- Si se declara **`t`** con el mismo tamaño que **`datos`**, se escribe fuera de rango

Producto de matrices: fila por columna



C[0][1] = fila 0 de A · columna 1 de B

= 1·8 + 2·10 + 3·12

= 8 + 20 + 36

= 64

Cada elemento de C: una fila de A por una columna de B, término a término, y se suma.

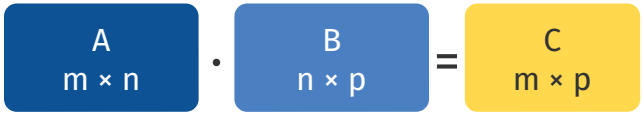
La fila de A y la columna de B deben tener el mismo largo:

2 × 3 · 3 × 2 → el resultado es **2 × 2**

Producto de matrices en código

```
const int M = 2, N = 3, P = 2;
float A[M][N] = { {1, 2, 3}, {4, 5, 6} };
float B[N][P] = { {7, 8}, {9, 10}, {11, 12} };
float C[M][P];

for (int i = 0; i < M; i++) {
    for (int j = 0; j < P; j++) {
        C[i][j] = 0;            // acumulador
        for (int k = 0; k < N; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```



$$C[i][j] = \Sigma k\ A[i][k] \cdot B[k][j]$$

- **i** recorre las filas de A y de C
- **j** recorre las columnas de B y de C
- **k** recorre el largo compartido n
- El acumulador **C[i][j]** arranca en 0

Errores frecuentes

! Índice fuera de rango

`datos[20][0]` no existe. Compila igual, pero lee basura o rompe el programa.

! Contar desde 1

Empezar en `i = 1` saltea la fila 0, y con `i <= 20` se pasa del final.

! Invertir i y j

`datos[j][i]` en lugar de `datos[i][j]`: lee otra variable u otra prueba.

! Acumulador sin inicializar

`float suma;` arranca con basura. Hay que darle valor inicial antes del `for`.

! División entera

`int / int` descarta decimales. Usar `float` en numerador o denominador.

! Faltan bibliotecas

`sqrt()` y `pow()` necesitan `#include <cmath>` al principio del programa.

Para el TP: preguntas antes de programar

a) ¿Qué dimensiones tiene la matriz de datos y cómo la muestran en pantalla?

b) Para la media de cada variable, ¿se recorre por fila o por columna? ¿Dónde guardan los 4 resultados?

b) La matriz estandarizada, ¿qué forma tiene? ¿Qué denominador usa el desvío muestral?

c) ¿De qué tamaño es la matriz de correlaciones? ¿Qué representa `m[p][q]`?

Punto de partida: `bombeo_matriz.cpp` ya tiene la matriz declarada e inicializada.